

Assignment 2 - Transformers

Alp Eren Gül

alp.gul@sabanciuniv.edu

1 Question

(a) Gradient calculation

We compute the gradient of the loss with respect to the anchor representation z . First, we rewrite the loss in a form that is easy to be differentiated: $\mathcal{L} = -\log p_0$, where p_0 is the softmax probability assigned to the positive sample $z_0 = z^+$. Since $p_0 = \frac{e^{s_0}}{Z}$ and $Z = \sum_{j=0}^K e^{s_j}$, taking the negative log gives $\mathcal{L} = -\log\left(\frac{e^{s_0}}{Z}\right) = -s_0 + \log Z$. This decomposition is useful because it separates the loss into a term which depends on the positive similarity ($-s_0$) and a term that depends on all similarities through the log-sum-exp normalizer ($\log Z$). Next, we differentiate $\mathcal{L}$ term by term with respect to z . The derivative of $-s_0$ gives $-\frac{\partial s_0}{\partial z}$, and the derivative of $\log Z$ gives $\frac{1}{Z} \frac{\partial Z}{\partial z}$ by the chain rule. We need to compute; $\frac{\partial Z}{\partial z}$, we use Z as it is a sum of exponentials $Z = \sum_{j=0}^K e^{s_j}$, so $\frac{\partial Z}{\partial z} = \sum_{j=0}^K e^{s_j} \frac{\partial s_j}{\partial z}$. Last step we differentiate each similarity score $s_j = \frac{z^\top z_j}{\tau}$ with z , this gives $\frac{\partial s_j}{\partial z} = \frac{1}{\tau} z_j$. Then we substitute these pieces to back and simplifying by $\frac{e^{s_j}}{Z} = p_j$, after all the calculations we obtain the desired gradient expression.

Solution:

Given We derive $\frac{\partial \mathcal{L}}{\partial z}$ by expanding $\mathcal{L}$ into terms whose derivatives are straightforward. Start from the definitions

$$s_j = \frac{z^\top z_j}{\tau}, \quad Z = \sum_{i=0}^K e^{s_i}, \quad p_j = \frac{e^{s_j}}{Z}.$$

Because $\mathcal{L} = -\log p_0$ and $p_0 = \frac{e^{s_0}}{Z}$, we write

$$\mathcal{L} = -\log\left(\frac{e^{s_0}}{Z}\right).$$

Use $\log\left(\frac{a}{b}\right) = \log a - \log b$:

$$\mathcal{L} = -(\log(e^{s_0}) - \log Z).$$

Use $\log(e^{s_0}) = s_0$:

$$\mathcal{L} = -(s_0 - \log Z) = -s_0 + \log Z.$$

Step 1: Differentiate $\mathcal{L} = -s_0 + \log Z$ w.r.t. z .

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial}{\partial z}(-s_0) + \frac{\partial}{\partial z}(\log Z) = -\frac{\partial s_0}{\partial z} + \frac{\partial}{\partial z}(\log Z).$$

Step 2: Differentiate $\log Z$ using the chain rule.

$$\frac{\partial}{\partial z}(\log Z) = \frac{1}{Z} \frac{\partial Z}{\partial z}.$$

So

$$\frac{\partial \mathcal{L}}{\partial z} = -\frac{\partial s_0}{\partial z} + \frac{1}{Z} \frac{\partial Z}{\partial z}.$$

Step 3: Differentiate $Z = \sum_{j=0}^K e^{s_j}$.

$$\frac{\partial Z}{\partial z} = \frac{\partial}{\partial z} \left(\sum_{j=0}^K e^{s_j} \right) = \sum_{j=0}^K \frac{\partial}{\partial z} (e^{s_j}).$$

Use $\frac{\partial}{\partial z} (e^{s_j}) = e^{s_j} \frac{\partial s_j}{\partial z}$:

$$\frac{\partial Z}{\partial z} = \sum_{j=0}^K e^{s_j} \frac{\partial s_j}{\partial z}.$$

Step 4: Differentiate each similarity score $s_j = \frac{z^\top z_j}{\tau}$. Since $\frac{\partial}{\partial z}(z^\top z_j) = z_j$,

$$\frac{\partial s_j}{\partial z} = \frac{\partial}{\partial z} \left(\frac{1}{\tau} z^\top z_j \right) = \frac{1}{\tau} \frac{\partial}{\partial z} (z^\top z_j) = \frac{1}{\tau} z_j.$$

In particular,

$$\frac{\partial s_0}{\partial z} = \frac{1}{\tau} z_0.$$

Step 5: Substitute Step 3 and Step 4 into the gradient expression.

First plug $\frac{\partial Z}{\partial z} = \sum_{j=0}^K e^{s_j} \frac{\partial s_j}{\partial z}$ into Step 2:

$$\frac{\partial \mathcal{L}}{\partial z} = -\frac{\partial s_0}{\partial z} + \frac{1}{Z} \sum_{j=0}^K e^{s_j} \frac{\partial s_j}{\partial z}.$$

Now substitute $\frac{\partial s_0}{\partial z} = \frac{1}{\tau} z_0$ and $\frac{\partial s_j}{\partial z} = \frac{1}{\tau} z_j$:

$$\frac{\partial \mathcal{L}}{\partial z} = -\frac{1}{\tau} z_0 + \frac{1}{Z} \sum_{j=0}^K e^{s_j} \frac{1}{\tau} z_j.$$

Factor out $\frac{1}{\tau}$:

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{1}{\tau} \left(-z_0 + \frac{1}{Z} \sum_{j=0}^K e^{s_j} z_j \right).$$

Step 6: Express the result using $p_j = \frac{e^{s_j}}{Z}$ and $z_0 = z^+$. Since $\frac{1}{Z} e^{s_j} = p_j$, we have

$$\frac{1}{Z} \sum_{j=0}^K e^{s_j} z_j = \sum_{j=0}^K p_j z_j.$$

Therefore,

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial z} &= \frac{1}{\tau} \left(\sum_{j=0}^K p_j z_j - z_0 \right) = \frac{1}{\tau} \left(\sum_{j=0}^K p_j z_j - z^+ \right). \\ \frac{\partial \mathcal{L}}{\partial z} &= \frac{1}{\tau} \left(\sum_{j=0}^K p_j z_j - z^+ \right). \end{aligned}$$

(b) Interpretation

(i) $(K+1)$ -(K+1)-way classifier view

In this part, we are going to interpret the InfoNCE objective as a standard multi-class classification problem with $(K+1)$ classes. The key idea is that the softmax probabilities p_j that already have the exact same form as a classifier's output probabilities produced from logits. So we are going to identify the logits with the similarity scores $s_j = \frac{z^\top z_j}{\tau}$, and we will identify the label with the index of the positive sample. After this identification, minimizing $\mathcal{L} = -\log p_0$ is exactly the same as minimizing cross-entropy loss for a classifier that should predict the positive class. We just need to specify: (1) what is the input, (2) what are the $(K+1)$ classes, and (3) which class is the correct label. Then we rewrite $\mathcal{L}$ in the standard cross-entropy form to make the equivalence explicit.

Step-by-step:

First, define the $(K+1)$ similarity scores (logits)

$$s_j = \frac{z^\top z_j}{\tau}, \quad j = 0, 1, \dots, K,$$

where $z_0 = z^+$ is the positive representation and $z_k = z_k^-$ for $k = 1, \dots, K$ are the negatives.

Second, define the softmax probabilities

$$p_j = \frac{e^{s_j}}{\sum_{i=0}^K e^{s_i}}, \quad j = 0, 1, \dots, K.$$

This is exactly the same functional form as a $(K+1)$ -class classifier output.

Third, interpret the classifier components:

Input: z (the anchor representation).

Classes: $j \in \{0, 1, \dots, K\}$ corresponding to candidates z_j (one positive and K negatives).

Correct label: $y = 0$ (the positive index).

Fourth, write the loss as cross-entropy with one-hot label $y = 0$:

$$\mathcal{L} = -\log p_0 = -\log \left(\frac{e^{s_0}}{\sum_{i=0}^K e^{s_i}} \right),$$

which is precisely the negative log-likelihood (cross-entropy) of predicting class 0 under a softmax model with logits $\{s_j\}_{j=0}^K$.

(ii) Geometric interpretation via the Gradient Descent update rule (push–pull)

We interpret the gradient geometrically by starting from the *gradient descent update rule*, rather than only inspecting the gradient vector. The reason is that the actual motion of the anchor representation z is determined by the *negative* gradient direction that appears in the update. So, we will (1) write the update rule, (2) substitute the InfoNCE gradient into it, (3) distribute the minus sign to separate the terms into attraction and repulsion, and (4) explain the resulting motion as a concrete push–pull mechanism in representation space.

Step 1: Write the gradient descent update rule.

Let $\eta > 0$ denote the learning rate. Gradient descent updates the anchor representation as

$$z_{new} \leftarrow z_{old} - \eta \nabla_z \mathcal{L}.$$

Step 2: Substitute the known InfoNCE gradient into the update rule.

We are given

$$\nabla_z \mathcal{L} = \frac{1}{\tau} \left(\sum_{j=0}^K p_j z_j - z^+ \right),$$

so plugging this into the update gives

$$z_{new} = z_{old} - \eta \cdot \frac{1}{\tau} \left(\sum_{j=0}^K p_j z_j - z^+ \right).$$

Step 3: Distribute the minus sign to expose attraction and repulsion terms.

Distribute the outer minus sign into the parentheses:

$$z_{new} = z_{old} + \frac{\eta}{\tau} \left(z^+ - \sum_{j=0}^K p_j z_j \right).$$

Now split the parentheses into two explicit contributions:

$$z_{new} = z_{old} + \underbrace{\frac{\eta}{\tau} z^+}_{\text{attractionterm}} - \underbrace{\frac{\eta}{\tau} \sum_{j=0}^K p_j z_j}_{\text{repulsionterm}}.$$

This algebraic step is the key: the $-$ sign in the update rule flips the sign of the gradient and turns the $-z^+$ part inside the gradient into a $+z^+$ contribution in the update (pull), while the $+\sum p_j z_j$ part becomes a $-\sum p_j z_j$ contribution (push).

Step 4: Physical/geometric meaning (how z moves in space).

The update says that the displacement

$$\Delta z = z_{new} - z_{old} = \frac{\eta}{\tau} \left(z^+ - \sum_{j=0}^K p_j z_j \right)$$

is a vector pointing from the *probability-weighted average of candidates* toward the *positive* representation. You can think of $\sum_{j=0}^K p_j z_j$ as a “center of mass” formed by all candidate vectors z_j , where each candidate pulls according to its softmax weight p_j . Because the update subtracts this weighted average, the anchor representation is pushed *away* from that center of mass. At the same time, due to the update adds z^+ , the anchor is pulled *toward* the positive example. So the anchor moves in the direction that simultaneously increases alignment with z^+ and decreases alignment with whatever candidates currently have large weights p_j . Importantly, those weights p_j are large exactly for the examples that are most similar to the anchor under the current model. That means the repulsion focuses on the “hard” negatives (the negatives that currently look most like the anchor), because they receive the largest softmax probabilities. This is why the mechanism is often described as “push–pull”: z is pulled toward the correct match z^+ , and pushed away from the set of competing representations, especially the ones that are confusingly similar. If you imagine forces, the positive vector applies an attractive force that tries to drag the anchor closer, while the competing examples form a repulsive field whose direction is the negative of their weighted average. The net motion is the vector difference between these two effects, it is scaled by $\frac{\eta}{\tau}$, which controls how far z moves in one step. As training proceeds and the anchor becomes more aligned with the positive, p_0 increases and the weighted average $\sum p_j z_j$ becomes dominated by the positive term, reducing the net displacement and stabilizing the representation. When the model is wrong or uncertain, some negatives get high p_j , and it is shifting the weighted average toward those negatives, and the update pushes the

anchor away from them while it is still pulling it toward z^+ . This combined behavior is exactly what the phrase “push-pull mechanism” refers to in geometric terms: a simultaneous attraction to the positive and repulsion from the probability-weighted competitor set.

(iii) Effect of temperature in the limits

In this part, we qualitatively analyze how τ changes the behavior of the gradient. There are two coupled effects of τ . First, τ appears explicitly as the prefactor $\frac{1}{\tau}$, which scales the magnitude of the gradient. Second, τ also appears inside the logits $s_j = \frac{z^\top z_j}{\tau}$, which changes the softness/sharpness of the softmax distribution p_j . Therefore, when τ becomes very small, the softmax becomes very peaky and it focuses on the single most similar sample. When τ becomes very large, the logits collapse toward zero and the softmax becomes nearly uniform. This is making the gradient both small and less discriminative. We summarize these behaviors for the two requested limits.

Step-by-step:

Limit 1: $\tau \rightarrow 0^+$.

As $\tau \rightarrow 0^+$, the logits $s_j = \frac{z^\top z_j}{\tau}$ become large in magnitude relative to each other, so the softmax becomes nearly one-hot. $p_j \rightarrow 1$ for the index j achieving the maximum similarity s_j , and $p_i \rightarrow 0$ for all $i \neq j$. Then the weighted sum $\sum_{j=0}^K p_j z_j$ is dominated by the single most similar representation (the “hardest” competitor). If the positive is already the most similar (i.e., $j = 0$), then $p_0 \rightarrow 1$ and

$$\sum_{j=0}^K p_j z_j \rightarrow z^+ \quad \Rightarrow \quad \frac{\partial \mathcal{L}}{\partial z} = \frac{1}{\tau} (z^+ - z^+) \rightarrow 0,$$

so the gradient vanishes because the classification is already confident and correct. If instead some negative is most similar (say $j = m \geq 1$), then $p_m \rightarrow 1$ and $p_0 \rightarrow 0$, so

$$\sum_{j=0}^K p_j z_j \rightarrow z_m^- \quad \Rightarrow \quad \frac{\partial \mathcal{L}}{\partial z} \approx \frac{1}{\tau} (z_m^- - z^+),$$

which points from the positive toward the hardest negative, and its magnitude is amplified by $\frac{1}{\tau}$. Small τ produces very sharp, hard-negative-focused updates and it can make optimization unstable if too small.

Limit 2: $\tau \rightarrow \infty$.

As $\tau \rightarrow \infty$, the logits satisfy $s_j = \frac{z^\top z_j}{\tau} \rightarrow 0$ for all j , so $e^{s_j} \rightarrow 1$. Hence the softmax becomes nearly uniform:

$$p_j = \frac{e^{s_j}}{\sum_{i=0}^K e^{s_i}} \rightarrow \frac{1}{K+1}.$$

Then

$$\sum_{j=0}^K p_j z_j \rightarrow \frac{1}{K+1} \sum_{j=0}^K z_j,$$

and the gradient becomes

$$\frac{\partial \mathcal{L}}{\partial z} \rightarrow \frac{1}{\tau} \left(\frac{1}{K+1} \sum_{j=0}^K z_j - z^+ \right).$$

Because of the explicit $\frac{1}{\tau}$ factor, the gradient magnitude goes to zero as $\tau \rightarrow \infty$, so learning becomes very slow. Also, p_j becomes uniform, the method is no longer focuses on hard negatives; it treats all candidates almost equally, reducing the contrastive “push-pull” selectivity.

2 Question

The link of the Notebook: colorlinks=true, linkcolor=blue, urlcolor=blue, citecolor=blue <https://colab.research.google.com/drive/1-pvnZD-dKITEi4Gddaebwtkxygny1ZxU?usp=sharing> for Google Colab notebook.